

In this lab, you will practice working with loops and lists. The TAs are here to help you. If you are stuck, ask for help!

QUESTION 1. COUNT EVENS

Here is a function that computes the sum of a list of numbers.

```
def sum_nums(L):  
    s = 0  
    for num in L:  
        s += num  
    return s
```

Write a function with the signature `def count_evens(L)` that returns the number of even integers in the list `L`. Assume `L` only contains integers.

QUESTION 2. LIST TO STRING

You can use `str()` to convert objects to strings.

```
s = str(42) # s contains the string "42"
```

In particular, you can obtain the string representation of a list `L` by using `str()`.

```
L0 = [1, 2, 3]  
L0_string = str(L0) # contains the string "[1, 2, 3]"
```

Without using `str()` with arguments that are lists (using it with arguments that are not lists is fine), write a function with the signature `list_to_str(lis)` which returns the string representation of the list `lis`. You may assume `lis` only contains integers.

Hint: recall that you can concatenate strings by adding them together.

QUESTION 3. LISTS ARE THE SAME

You can compare lists using the `==` operator.

```
L1 = [1, 2, 3]  
L2 = [4, 5, 6]  
L3 = [1, 2, 3]  
print(L1 == L2) # False  
print(L1 == L3) # True
```

Without using the `==` operator to compare lists (you can still compare individual elements of the lists), write a function `lists_are_the_same(list1, list2)` which returns `True` if and only if `list1` and `list2` contain the same elements in the same order. Note that you will need to use a loop (either `while` or `for`).

QUESTION 4. INPUT LOOP

Write a program that repeatedly asks the user for names using the `input()` function, and then outputs the list of all the names before the special name END is entered. An example of an interaction would be:

```
Enter a name: Alice
Enter a name: Bob
Enter a name: Charlie
Enter a name: Dave
Enter a name: Emily
Enter a name: END
The names are: Alice, Bob, Charlie, Dave, Emily
```

To store the names in the program, you can use a string. For example, the string might be "" at first, then "Alice", then "Alice, Bob", etc.

QUESTION 5. PI APPROXIMATION WITH SIG FIGS

Recall that in Lab 3, you used the Leibniz formula to approximate π .

Now we would like to figure out how many terms in the summation we need to add up in order to approximate π to n significant digits. Write a function that returns the number of the terms required to obtain an approximation of π using the Leibniz formula that agrees with the actual value of π to n significant digits (i.e., the first n digits and the same in π and in the approximation).

The best approximation of π using a float is available in `math.pi` (execute `import math` to be able to use it.)

Part of your job is to figure out whether two numbers agree to n significant digits. To figure that out, for a float x , consider what `int(x*(10**n))` means.

QUESTION 6. GREATEST COMMON DIVISOR USING EXHAUSTIVE SEARCH

In this question, you will write a function with the signature `gcd(n, m)` which computes the greatest common divisor of the positive integers n and m . The greatest common divisor of n and m is a number d such that n is divisible by d and m is divisible by d .

Note that k is divisible by d if and only if the remainder of the division of k by d is 0, i.e., `k % d == 0`. There is an efficient algorithm for finding the greatest common divisor (more on this later), but in this question, you should use exhaustive search: trying every possible answer until you find the right one.

Implement both of the approaches on the following page.

Part (a)

Write the function by trying every possible divisor from 1, 2, 3, ..., etc. (What is the largest guess you can try?). Keep track of the largest guess for the greatest common divisor that worked so far. Update a variable every time the divisor divides both n and m . Return the latest guess that worked.

```
def gcd(n, m):  
    """Return the greatest common divisor of n and m. Use  
    exhaustive search, trying divisors from 1 up to the maximum  
    possible."""  
    # Your code here
```

Part (b)

The approach from part (a) is inefficient in that we always have to try all the possible guesses every time. If we tried the largest guess first and it worked, we would not need to try smaller guesses (**explain why to a TA**). Use a loop (either `while` or `for`) to try all the possible guesses, from the largest to the smallest, and use an early return technique to return from the function once you know the answer.

```
def gcd_efficient(n, m):  
    """Return the greatest common divisor of n and m. Use  
    exhaustive search, trying divisors from the largest down to  
    1. Return immediately when the GCD is found."""  
    # Your code here
```

QUESTION 7. EUCLID'S ALGORITHM (CHALLENGE)

You may demo Questions 1-6 to a TA to receive your lab mark before attempting this question.

A more efficient way of finding the greatest common divisor than what we suggested in the previous question is Euclid's algorithm. An explanation and example of the algorithm can be found at the following webpage:

- <https://www.cut-the-knot.org/blue/Euclid.shtml>

Implement and test Euclid's algorithm, and compare the number of steps it takes to the number of steps that the exhaustive search takes.